PROJECT AND TEAM INFORMATION

Project Title

ResourceX – Intelligent Adaptive Resource Manager

Student / Team Information

Team Name: *Team #*	*ResourceX*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	Bhatt, Nitish ,24021739, nitisshbhatt3131@gmail.com 
Team member 2 *(Last Name, name: student ID: email, picture):*	Chauhan,Rajveer,240211139, rajveer@455677@gmail.com 
Team member 3 *(Last Name, name: student ID: email, picture):*	Gulia,Muskan,240222164, mgulia185@gmail.com 
Team member 4 *(Last Name, name: student ID: email, picture):*	Vibhanshu,240211269, Vibhanshusiwach87@gmail.com 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

Modern laboratories, organizations, and computing environments may contain many computers with different CPU, memory, workload, and availability conditions. These machines are often managed independently, which can result in some computers being underutilized while others are overloaded.

ResourceX aims to provide an intelligent distributed resource management platform that allows authorized computers to cooperate as a computing resource pool. Lightweight agents installed on worker computers continuously report CPU usage, memory usage, running processes, availability, and execution information to a central management system. When a user submits a computational job, ResourceX evaluates the job requirements together with the current and historical condition of available workers. Instead of using only a fixed scheduling rule, the system adaptively selects a suitable worker and learns from previous execution results.

The project addresses inefficient resource utilization, workload imbalance, unreliable worker availability, and limited visibility of distributed resources while demonstrating practical Operating System, DBMS, distributed-system, and intelligent-system concepts.

State of the Art / Current Solution (1 pt)

Traditional operating systems use scheduling algorithms such as First-Come, First-Served (FCFS), Round Robin, and Priority Scheduling to manage processes on individual machines. These approaches are effective for local process management but do not directly solve resource selection across multiple independent computers.

Distributed and cloud computing platforms provide advanced resource management. Systems such as cluster schedulers and container orchestration platforms can distribute workloads across multiple machines according to resource requirements and availability. However, scheduling decisions may still depend heavily on predefined policies and current resource information.

Research on systems such as MapReduce and Mesos demonstrates the importance of distributed scheduling, fault handling, resource sharing, and workload distribution. ResourceX builds on these ideas while focusing on a smaller academic prototype that additionally considers historical worker performance, worker reliability, workload trends, and adaptive resource selection.

Project Goals and Milestones (2 pts)

Project Goals

- Develop ResourceX as an intelligent platform for monitoring and managing computing resources.
- Allow authorized worker computers to participate in a shared resource pool.
- Monitor CPU, RAM, processes, availability, and workload information.
- Implement resource-aware and adaptive job scheduling.
- Store historical resource and execution information in PostgreSQL.
- Detect unavailable workers using heartbeat monitoring.
- Demonstrate OS concepts such as process management, scheduling, synchronization, resource allocation, load balancing, and failure handling.
- Provide a web-based dashboard for monitoring and administration.

Milestone 1 – Requirements and Literature Study

Identify functional requirements, study existing resource-management and scheduling systems, and finalize the project scope.

Milestone 2 – System Architecture and Database Design

Design the overall architecture, communication interfaces, database schema, worker-agent structure, and resourcemanagement workflow.

Milestone 3 – Backend Development

Develop the central server, authentication, worker registration, job management, APIs, and database integration.

Milestone 4 – ResourceX Agent

Develop the lightweight worker agent for collecting CPU, memory, process, availability, and execution information.

Milestone 5 – Scheduling and Adaptive Resource Management

Implement resource-aware scheduling, worker selection, heartbeat monitoring, and adaptive allocation.

Milestone 6 – Intelligent Decision Module

Develop the module that combines current resource conditions with historical performance information.

Milestone 7 – Frontend and Dashboard

Develop the web interface for job submission, worker monitoring, system statistics, and administration.

Milestone 8 – Integration, Testing and Evaluation

Integrate all components, test different workload conditions, evaluate performance, fix failures, and prepare the final documentation.

Project Approach (3 pts)

1. System Architecture

- Use a modular client-server architecture.
- A central ResourceX server will coordinate multiple authorized worker computers.
- Each worker computer will run a lightweight ResourceX Agent.

2. Resource Monitoring

- The ResourceX Agent will collect CPU utilization, memory usage, running processes, availability, and execution information.
- Agents will periodically send monitoring information to the central backend.

3. Backend and Database

- FastAPI will be used to implement backend APIs.
- PostgreSQL will store users, workers, jobs, resource history, execution records, heartbeats, and system logs.
- REST-based communication will be used between agents, frontend, and backend.

4. Adaptive Scheduling

- The scheduler will compare job requirements with available worker resources.
- Current resource information and historical execution performance will be considered when selecting workers.
- The system will update future decisions based on observed execution results.

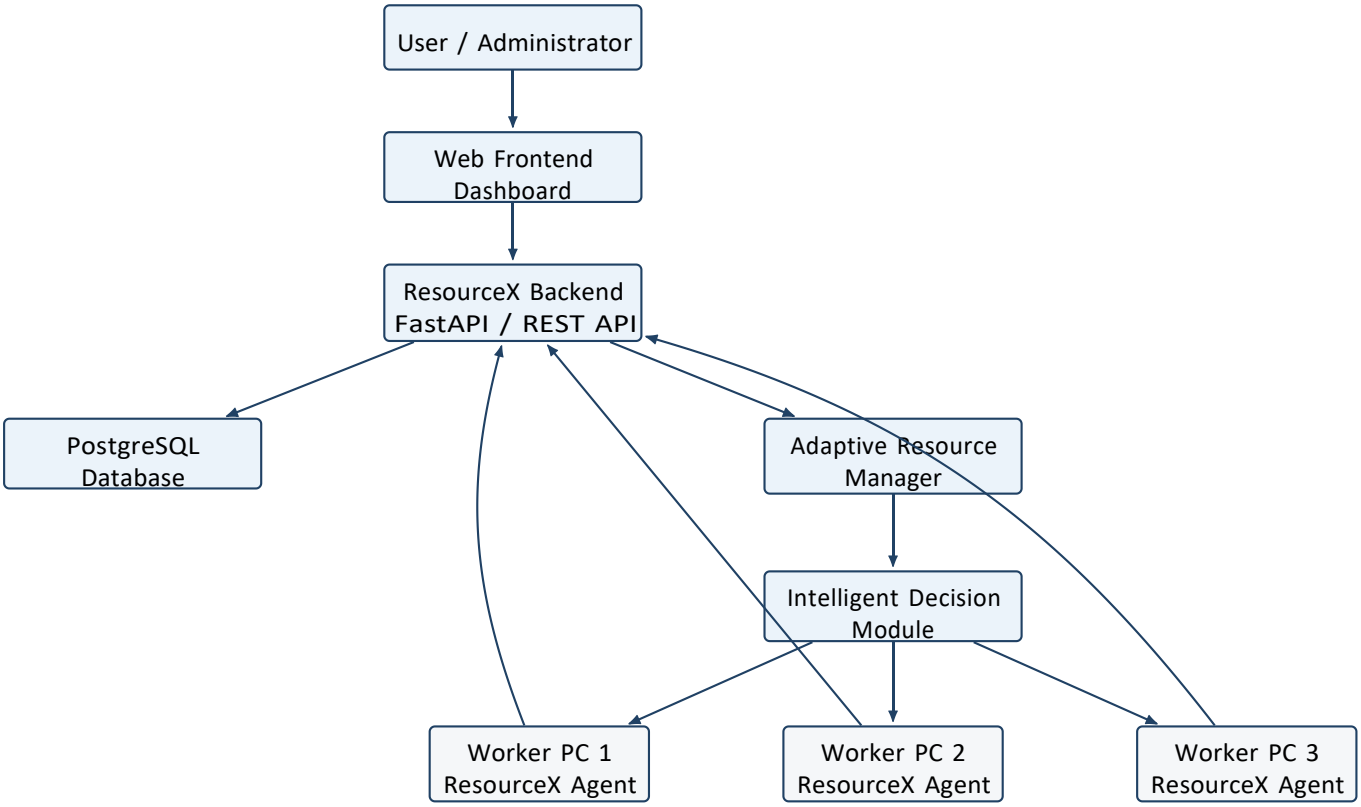
5. Operating-System Integration

- OS-level concepts such as process management, scheduling, resource allocation, synchronization, load balancing, and failure detection will be demonstrated.
- C++ can be used for core OS-oriented resource-management components where required.

6. Frontend and Evaluation

- A web dashboard will display workers, resource utilization, jobs, scheduling decisions, execution results, and failures.
- The prototype will initially be tested using computers connected through a local network and simulated worker workloads.

System Architecture (High Level Diagram) (2 pts)



Architecture Explanation:

ResourceX follows a modular distributed architecture. Users interact with the system through a web frontend for submitting jobs and monitoring system status. The FastAPI backend manages users, jobs, worker registration, APIs, and communication between the different components.

The PostgreSQL database stores worker information, job records, resource history, execution results, heartbeat information, and system logs. The Adaptive Resource Manager is responsible for scheduling and assigning jobs to suitable worker computers.

The Intelligent Decision Module uses current resource conditions and historical performance information to assist worker selection. Each worker computer runs a ResourceX Agent that collects CPU, memory, process, availability, and execution information and sends the information back to the backend.

The architecture separates the main responsibilities of the system while keeping communication between the components simple and manageable.

Project Outcome / Deliverables (1 pt)

.

Final Outcome

The final outcome will be a working prototype of **ResourceX – Intelligent Adaptive Resource Manager**. The completed system will provide a centralized platform where authorized computers can participate in a shared computing resource pool.

The system will be able to:

- Register and monitor multiple worker computers.
- Collect CPU, memory, process, availability, and workload information.
- Accept computational jobs from authorized users.
- Select a suitable worker based on current resource availability and historical performance.
- Allocate and execute jobs on selected worker computers.
- Detect unavailable workers using heartbeat monitoring.
- Store resource and execution history in PostgreSQL.
- Display worker status, jobs, resource usage, and execution information through a web dashboard.
- Demonstrate operating-system concepts such as scheduling, process management, resource allocation, synchronization, load balancing, and failure handling.

Therefore, the final deliverable will be an integrated prototype consisting of the backend server, ResourceX Agents, database, adaptive scheduling module, intelligent decision module, web dashboard, testing results, and technical documentation.

Assumptions

- Worker computers are authorized to participate in the ResourceX system.
- Each participating computer has the ResourceX Agent installed.
- Worker computers have a network connection to the central server.
- Users are authenticated before submitting computational jobs.
- The prototype will initially operate in a controlled academic or laboratory environment.
- The system will collect only system-level information required for resource management.
- Worker computers may have different hardware configurations and resource conditions.

References

1. Silberschatz, A., Galvin, P. B., and Gagne, G., *Operating System Concepts*, Wiley.
2. Tanenbaum, A. S. and Bos, H., *Modern Operating Systems*, Pearson.
3. Tanenbaum, A. S. and van Steen, M., *Distributed Systems: Principles and Paradigms*, Pearson.
4. Ramakrishnan, R. and Gehrke, J., *Database Management Systems*, McGraw-Hill.
5. PostgreSQL Global Development Group, *PostgreSQL Documentation*. https://www.postgresql.org/docs/
6. FastAPI, *FastAPI Documentation*. https://fastapi.tiangolo.com/
7. Apache Software Foundation, *Apache Hadoop MapReduce Documentation*. https://hadoop.apache.org/docs/
8. Apache Mesos, *Apache Mesos Documentation*. https://mesos.apache.org/documentation/latest/
9. Kubernetes, *Kubernetes Documentation – Scheduling*. https://kubernetes.io/docs/concepts/scheduling-eviction/ kube-scheduler/
10. Dean, J. and Ghemawat, S., *MapReduce: Simplified Data Processing on Large Clusters*, Communications of the ACM.